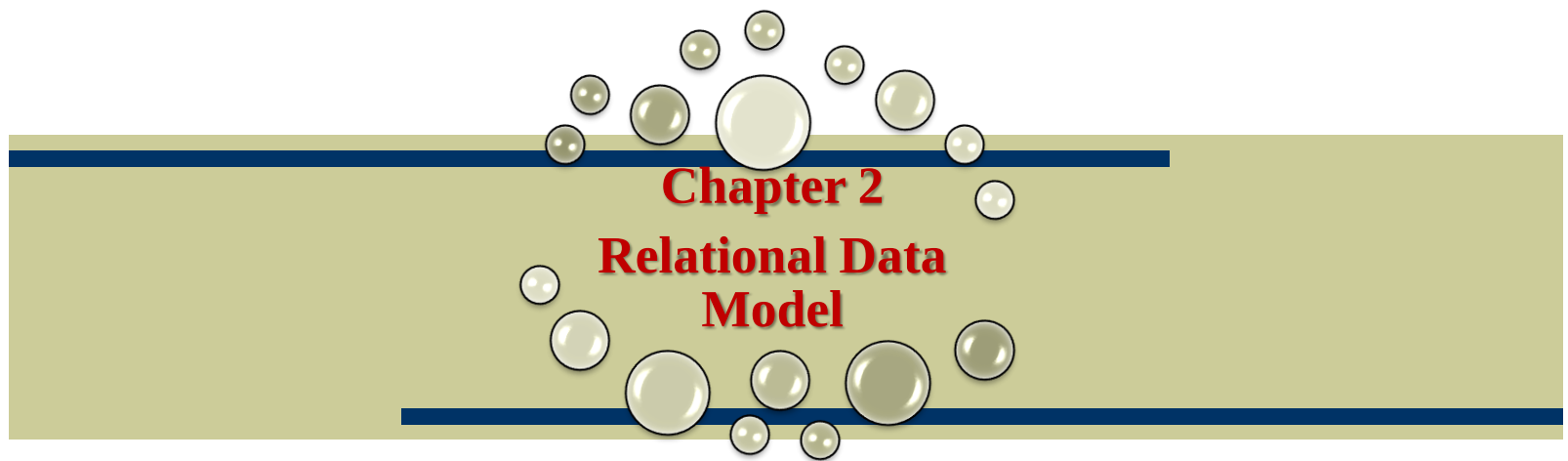




2.6 Data Integrity Constraints:

Types of Data Integrity Constraints:

I/O Constraint – Primary Key, Foreign Key, Unique Key Constraint.

Business Rule Constraint – Not Null and Check Constraint.

	Prepared By
	Mohammad Ali
	Lecturer (Selection Grade)
	M. H. Saboo Siddik Polytechnic, Mumbai

Data Integrity Constraints

- ◆ The term data integrity simply means that the data stored in the table is **valid / authentic**.
- ◆ There are different types of data integrity, often referred to as constraints.

NOT NULL Constraint

- ◆ A NOT NULL constraint means that a data row must have a value for the column specified as NOT NULL.
- ◆ A fairly standard practice is to assign each constraint a *unique constraint name*.
- ◆ If constraints are not named, then Oracle assigns meaningless system-generated names to each constraint.

Example:

```
Create table employee  
(ename VARCHAR2(25) CONSTRAINT nn_ename NOT NULL,  
salary number(8,2) CONSTRAINT nn_sal NOT NULL) ;
```

PRIMARY KEY Constraint

- ◆ Each table must normally contain a column or set of columns that uniquely identifies rows of data that are stored in the table.
- ◆ This column or set of columns is referred to as the *primary key*.
- ◆ If a table requires two or more columns in order to identify each row , the primary key is termed a *composite* primary key.

Example:

Create table emp1

```
(empno number(4) CONSTRAINT pk_emp1 PRIMARY KEY,  
ename VARCHAR2(25) CONSTRAINT nn_ename_emp1 NOT  
NULL, salary number(8,2) CONSTRAINT nn_sal_emp1  
NOT NULL);
```

CHECK Constraint

- ◆ Sometimes the data values stored in a specific column must fall within some acceptable range of values.
- ◆ A CHECK constraint is used to enforce this data limit.

Example:

Create table emp2

```
(empno number(4) CONSTRAINT pk_emp2 PRIMARY KEY,  
ename VARCHAR2(25) CONSTRAINT nn_ename NOT NULL,  
salary number(8,2) CONSTRAINT ck_salary  
CHECK(salary <= 500000)) ;
```

UNIQUE Constraint

- ◆ Sometimes it is necessary to enforce uniqueness for a column value that is not primary key column.
- ◆ The *UNIQUE* constraint can be used to enforce this rule and Oracle rejects any rows that violate the constraint.
- ◆ For example, No two employees can be assigned same mobile number.

Example:

Create table emp3

```
(empno number(4) CONSTRAINT pk_emp3 PRIMARY KEY,  
ename VARCHAR2(25) CONSTRAINT nn_ename NOT NULL,  
sal number(8) CONSTRAINT ck_sal CKECK(sal<= 50000)  
mob_no number(10) CONSTRAINT un_mob_no UNIQUE );
```

FOREIGN Keys

- ◆ Normally, data rows in one table are related to data rows in other tables.
- ◆ The *department* table will store information about departments within the organization.
- ◆ Each department has a unique, 2-digit department number, department name, and location.

Example:

```
CREATE TABLE dept
(deptno NUMBER(2) CONSTRAINT pk_dept PRIMARY KEY,
dname VARCHAR2(20) CONSTRAINT nn_dname NOT NULL
location varchar29(10));
```

Identifying FOREIGN Keys

- ◆ Normally, in an organization, at a given time, an employee is assigned to a single department.
- ◆ In order to link rows in the *employee* table to rows in *dept* table we need to introduce a new type of constraint, the **FOREIGN KEY** constraint.
- ◆ **Foreign keys (FKs)** are columns in one table that reference **primary key (PK)** values in **another** or in the **same table**.
- ◆ Lets relate employee rows to the PK column named *deptno* in the **dept** table.
- ◆ The value stored to the **emp_deptno** column for any given *employee* row must match a value stored in the *deptno* column in the **dept** table.

Identifying FOREIGN Keys

```
CREATE TABLE employee
(emp_no number(5)
        CONSTRAINT pk_employee PRIMARY KEY,
emp_name  VARCHAR2(25)
        CONSTRAINT nn_emp_name NOT NULL,
emp_dob DATE,
emp_salary NUMBER(8,2)
        CONSTRAINT ck_emp_salary
        CHECK (emp_salary <= 55000),
mob_no number(10) CONSTRAINT un_mob_no UNIQUE,
emp_deptno NUMBER(2),
CONSTRAINT fk_emp_dpt FOREIGN KEY (emp_deptno)
REFERENCES dept(deptno) ON DELETE SET NULL);
```

MAINTAINING REFERENTIAL INTEGRITY

- ◆ FOREIGN KEY constraints are also referred to as referential integrity constraints, and assist in maintaining database integrity.
- ◆ Referential integrity stipulates that values of a foreign key must correspond to values of a primary key in the table that it references. But what happens if the primary key values change or the row that is referenced is deleted?
- ◆ Several methods exist to ensure that referential integrity is maintained.
- ◆ ON DELETE SET NULL clause can be used to specify that the value of *emp_dpt_number* should be set to null if the referenced *department* row is deleted.
- ◆ There are additional referential integrity constraints that can be used to enforce database integrity.

MAINTAINING REFERENTIAL INTEGRITY

On Update (Delete) Restrict	Any update/delete made to the <i>department</i> table that would delete or change a primary key value will be rejected unless no foreign key references that value in the <i>employee</i> table. This is the <i>default</i> constraint in Oracle.
On Update (Delete) Cascade	Any update/delete made to the <i>department</i> table should be cascaded through to the <i>employee</i> table.
On Update (Delete) Set Null	Any values that are updated/deleted in the <i>department</i> table cause affected columns in the <i>employee</i> table to be set to null.

MAINTAINING REFERENTIAL INTEGRITY

Specify the behavior for child tuples when a parent tuple is modified.

Action to take if referential integrity is violated:

- SET NULL - Child tuples foreign key is set to NULL - Orphans.
- SET DEFAULT - Set the value of the foreign key to some default value.
- CASCADE - Child tuples are updated (or deleted) according to the action take on the parent tuple.

Example - Primary key

- ◆ **CREATE TABLE order_header (**
order_number NUMBER(10,0) NOT NULL,
order_date DATE,
sales_person VARCHAR(25),
bill_to VARCHAR(35),
bill_to_address VARCHAR(45),
bill_to_city VARCHAR(20),
bill_to_state VARCHAR(2),
bill_to_zip VARCHAR(10),
PRIMARY KEY (order_number));

Example - Foreign key

- ◆ **CREATE TABLE** order_items (
order_number NUMBER(10,0) NOT NULL,
line_item NUMBER(4,0) NOT NULL,
part_number VARCHAR(12) NOT NULL,
quantity NUMBER(4,0),
PRIMARY KEY (order_number, line_item),
FORIEGN KEY (order_number)
 REFERENCES order_header (order_number),
FOREIGN KEY (part_number)
 REFERENCES parts (part_number));

Example

Table Level Constraint

```
CREATE TABLE order_items (  
    order_number      NUMBER(10,0) NOT NULL,  
    line_item         NUMBER(4,0)  NOT NULL,  
    part_number       VARCHAR(12)  NOT NULL,  
    quantity          NUMBER(4,0),  
    PRIMARY KEY (order_number, line_item),  
    FOREIGN KEY (order_number)  
        REFERENCES order_header (order_number)  
        ON DELETE SET DEFAULT  
        ON UPDATE CASCADE,  
    FOREIGN KEY (part_number)  
        REFERENCES parts (part_number)  
);
```

Example

Table Level Constraint with name

```
CREATE TABLE order_header (  
  order_number NUMBER(10,0) NOT NULL,  
  order_date DATE,  
  sales_person VARCHAR(25),  
  bill_to VARCHAR(35),  
  bill_to_address VARCHAR(45),  
  bill_to_city VARCHAR(20),  
  bill_to_state VARCHAR(2),  
  bill_to_zip VARCHAR(10),  
  CONSTRAINT order_header_pk  
  PRIMARY KEY (order_number) );
```


Example : Creating Table from another table & Dropping it

- ◆ `CREATE TABLE emp_department_1 AS SELECT fname, lname, bdate FROM employee WHERE dno = 1 ;`
- ◆ `create table high_pay_emp as select * from employee where salary > 50000`
- ◆ `Drop table <table_name>`
- ◆ `Drop table high_pay_emp`